📘 RAG SYSTEM DESIGN & IMPLEMENTATION

A Complete Language-Neutral Guide for Beginners to Professionals

TABLE OF CONTENTS

1. Executive Summary

2. Understanding the Fundamentals of RAG

2.1 Why RAG Exists
2.2 Retrieval-Augmented Generation workflow
2.3 Key Terminology

3. What You Must Know Before Building a RAG System

3.1 Same Embedding Model Requirement
3.2 Tokenization
3.3 Choosing Chunk Size
3.4 Choosing a Vector Database

4. Tools, Libraries & Frameworks in Different Languages

4.1 Python Ecosystem
4.2 Java Ecosystem
4.3 JavaScript / Node.js Ecosystem
4.4 C# / .NET Ecosystem
4.5 Cloud-Native RAG Tools

5. End-to-End Step-by-Step RAG Workflow

5.1 Document Collection
5.2 Chunking
5.3 Embedding
5.4 Vector Database Storage
5.5 Retriever Design
5.6 LLM Integration
5.7 Backend API Development
5.8 Frontend Chat UI
5.9 Deployment Strategy

6. Security, Privacy & Governance

6.1 PII Masking
6.2 Secure Embedding Storage
6.3 Role-Based Access Control (RBAC)
6.4 Audit Logging

6.5 Document Isolation
6.6 Identity Integration (OAuth/OIDC)

7. Scaling a RAG System

7.1 Why Scaling Matters
7.2 Horizontal Scaling of Retrievers
7.3 Distributed Vector Databases
    7.3.1 Milvus / Zilliz
    7.3.2 Pinecone
    7.3.3 Weaviate Cloud
7.4 Sharding Strategies
7.5 Cache-Based Retrieval Acceleration
7.6 Index Optimization
7.7 Scaling the LLM Layer

8. Testing a RAG System

8.1 LangSmith
8.2 Langfuse + Ragas
8.3 Phoenix
8.4 TruLens
8.5 Promptfoo
8.6 Helicone
8.7 Weights & Biases

9. Prompt Engineering for RAG Development

9.1 Full RAG Application Generation Prompt
9.2 Vector Database Index Prompt
9.3 Chat History Prompt
9.4 Deployment Script Prompt
9.5 RBAC & Governance Prompt
9.6 UI Security Prompt

10. Banking Domain Use Case

10.1 Loan Policy RAG
10.2 Interest Rate Retrieval
10.3 KYC Compliance
10.4 Metadata-Enriched Retrieval
10.5 Customer Attention & Verification RAG

11. Troubleshooting Guide

11.1 Mismatched Embeddings
11.2 Poor Chunking
11.3 Hallucinations

11.4 Memory Overflow
11.5 Index Corruption
11.6 Slow Retrieval
11.7 Master Troubleshooting Prompt

12. Final Recommendations

12.1 Embedding Selection
12.2 Metadata Filtering
12.3 Chunk Size Consistency
12.4 Hybrid Search Strategy
12.5 Logging Practices
12.6 Index Versioning

---

1. Executive Summary

Retrieval-Augmented Generation (RAG) is an AI architecture that enhances Large Language Models (LLMs) using external knowledge sources—primarily documents stored in a vector database. Unlike traditional systems where the model must "memorize" all information, RAG gives the LLM the ability to fetch authoritative, domain-specific content at runtime.

This manual is a complete, step-by-step foundational guide that covers:

- What you must understand before building RAG
- Tools & libraries in all major languages
- Architecture breakdown
- Document ingestion workflows
- Chunking, embedding, vector storage
- Retriever design best practices
- LLM integration patterns
- API + UI development
- Deployment strategies
- Security considerations
- Prompt engineering guide for generating RAG apps using GenAI tools
- End-to-end example prompts
- Banking domain example use cases

---

2. Understanding the Fundamentals of RAG

2.1 Why RAG Exists

LLMs are powerful but have limitations:

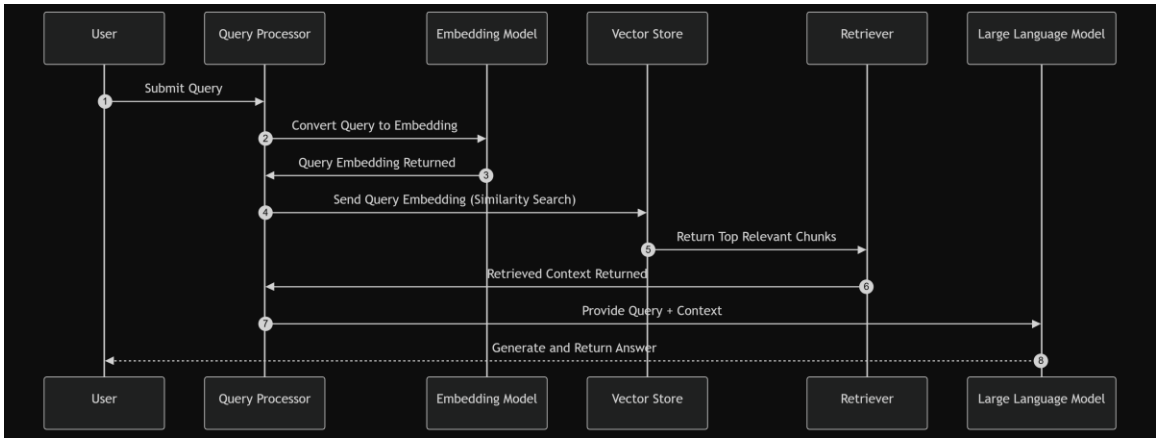
- They cannot store large domain knowledge internally
- They hallucinate when lacking context
- They cannot access private or recent documents

RAG solves this.

Core Concept

1. Convert documents into embeddings
2. Store them in a vector database
3. Convert user query into embeddings
4. Retrieve the most relevant document chunks
5. Pass them + the query to the LLM
6. Generate grounded answers

2.2 Retrieval-Augmented Generation workflow



2.3 Key Terminology

- **Embedding** — numerical vector representing semantic meaning
- **Chunking** — splitting documents into manageable pieces
- **Retriever** — fetches relevant chunks
- **Vector Database** — stores embeddings efficiently for similarity search
- **RAG Pipeline** — the full flow: retrieve → augment → generate

---

3. What You Must Know Before Building a RAG System

3.1 The Most Important Rule: SAME Embedding Model

You *must* use the **same embedding model** for:

- Indexing your documents
- Embedding user queries

If inconsistent:

- Similarity scores fail
- Retrieval quality collapses
- RAG becomes useless

3.2 Understanding Tokenization

Tokenization determines:

- Chunk sizes
- Context window usability
- LLM cost
- Input length limits

Different LLMs use different tokenizers.

3.3 Choosing Chunk Size

General rules:

- **Small chunk → precision**
- **Large chunk → richer context**

Recommended:

- 300–1200 tokens
- 100–150 overlap

3.4 Choosing a Vector Database

Key Factors for Choosing a Vector Database

1. Dataset Size

- **Small (≤1M vectors):** Chroma, FAISS
- **Medium/Large (1M–50M):** Milvus, Weaviate
- **Huge (50M+):** Pinecone, Zilliz Cloud

2. Deployment Preference

- **Local:** FAISS, Chroma, Milvus-local
- **Managed Cloud:** Pinecone, Weaviate Cloud, Azure AI Search

3. Performance & Latency

- Should support **ANN search** (HNSW/IVF)
- Goal: **<200ms** retrieval time
- Good for production: Pinecone, Weaviate, Milvus

4. Metadata & Filtering

Choose DBs that support:

- Metadata filters (page, section, category)
- Hybrid search (semantic + keyword)

Best: Pinecone, Weaviate, Azure AI Search

5. Cost

- **Free/Open-source:** FAISS, Chroma, Milvus
- **Paid/Managed:** Pinecone, Weaviate Cloud, Azure AI Search

6. SDK Support (Java-Friendly)

- Best for Spring Boot / LangChain4j:
  Pinecone, Weaviate, Milvus

7. Reliability & Backup

- Cloud DBs provide:
  Auto backup → DR → Multi-region replication

8. Security & Compliance

Look for:

- Encryption
- RBAC
- Audit logs
- SOC2/GDPR compliance

Ideal for banking: **Azure AI Search, Pinecone, Zilliz Cloud**

---

4. Tools, Libraries & Frameworks in Different Languages

Different programming languages offer their own ecosystems and strengths when building RAG applications. **Python** provides the richest and most mature tooling through frameworks like LangChain, LlamaIndex, Haystack, and strong support for vector databases, making it ideal for rapid prototyping, research workflows, and experimentation. **Java**, through LangChain4j, Pinecone/Weaviate clients, and Spring Boot integration, is highly suited for enterprise-grade, production-ready systems—especially in banking, insurance, and regulated industries. **JavaScript/Node.js** offers lightweight, full-stack integration using LangChain.js and LlamaIndex.js, making it useful for building fast web-based chat interfaces. **C#/.NET** developers can leverage Semantic Kernel and Azure Cognitive Search SDK to build deeply integrated RAG apps within Microsoft ecosystems. The language choice depends on the team's familiarity, availability of libraries, performance needs, and whether the application targets web, enterprise, or experimentation scenarios.

4.1 Python

LangChain
LlamaIndex
Haystack
FAISS
ChromaDB
Pinecone Client

Example Use Cases
– Rapid prototyping of RAG pipelines
– Research workflows & experimentation
– Building agent-based or orchestration-heavy systems
– Local/offline RAG using open-source models

4.2 Java

LangChain4j
Pinecone Java Client
Weaviate Java SDK
Milvus Java SDK

Example Use Cases
– Enterprise backends (banking, insurance, fintech)
– Production-grade microservices with Spring Boot
– RAG APIs deployed in secure corporate environments
– Systems requiring strong typing, stability & governance

4.3 JavaScript / Node.js

LangChain.js
LlamaIndex.js
Pinecone JS SDK
Weaviate JS Client

Example Use Cases
– Web-based RAG chat interfaces
– Serverless RAG apps (Vercel / Netlify / Cloudflare Workers)
– Lightweight data pipelines for embeddings
– Frontend + backend full-stack demos

4.4 C# / .NET

Semantic Kernel
Azure Cognitive Search SDK
Azure OpenAI .NET SDK

Example Use Cases
– RAG apps inside the Microsoft ecosystem
– Enterprise internal tools (SharePoint, Teams, Office add-ins)
– Highly regulated workloads needing Azure compliance
– Backend APIs using ASP.NET Core

4.5 Cloud Tools

Amazon Q Developer
Azure AI Studio RAG Pipelines
Google Vertex AI Grounding
AWS Bedrock Knowledge Bases

Example Use Cases
– End-to-end RAG pipelines without managing infrastructure
– Auto-generated code for ingestion, retrieval, and chat APIs
– Enterprise deployment with built-in security & monitoring
– High-scale search with managed vector databases

---

5. End-to-End Step-by-Step RAG Workflow

Step 1: Collect and Prepare Documents

Common formats:

- PDF
- DOCX
- CSV
- HTML pages

Preprocessing:

- Remove page numbers
- Normalize spacing
- Remove headers/footers
- Extract clean text

Step 2: Chunking Process

Goals of Chunking

- Provide meaningful semantic pieces
- Avoid overwhelming the LLM

Strategies:

- Recursive text splitting
- Semantic chunking
- Headline-based chunking
- Page-level splitting for structured PDF manuals

Step 3: Embedding Documents

Embedding model options:

- **OpenAI / Azure**: text-embedding-3-large
- **Amazon Titan Embeddings**
- **Google Gecko Embeddings**
- **Local Models**: BGE-large, E5-large

Properties of embeddings:

- Dimensionality
- Vector distance metrics
- Cosine vs dot product

Step 4: Vector Database Storage

Vector DB requirements:

- Fast similarity search
- Metadata filtering
- Scalability
- Persistence

Metadata to store:

- Document title
- Section name
- Page number
- Timestamp

Step 5: Designing the Retriever

Retriever choices:

- Vector similarity
- Hybrid search (keyword + vector)

- Metadata-aware search
- Reranking with cross-encoders

Step 6: LLM Integration

Models commonly used:

- GPT-4 Turbo (Azure)
- Amazon Bedrock Claude Models
- Google Gemini Pro
- Local Llama-3 based models

Input to LLM:

- User query
- Retrieved chunks
- System prompt

Step 7: Building the Backend APIs

Typical endpoints:

- POST /ingest
- POST /chat
- GET /health
- GET /stats

Step 8: Building a Frontend Chat UI

Features:

- Conversation history
- Streaming responses
- Error handling

- Document source citations

Step 9: Deployment Strategy

AI can strengthen the deployment pipeline by automatically generating test datasets and evaluating your RAG system before each release. Platforms like **LangSmith** help run AI-generated questions through your pipeline and score answer relevance, accuracy, and retrieval quality, allowing CI/CD to block deployments if the model performance drops. This ensures high reliability before going to production

Options:

- Docker Compose
- Kubernetes
- Azure App Service
- AWS ECS/EKS
- Serverless functions

---

6. Security, Privacy & Governance

A production-grade RAG system must be designed with strong security, privacy controls, and governance policies to protect sensitive data at every stage of ingestion, storage, retrieval, and response generation. Security is especially critical because vector embeddings, retrieved documents, and user prompts may contain confidential or regulated information. Implementing proper access controls, auditing, and data-isolation mechanisms ensures that the system remains trustworthy, compliant, and protected against unauthorized access or data leakage.

PII Masking

All personally identifiable information (PII) should be detected and masked during ingestion and retrieval. Automated scanners or AI-assisted validators can inspect documents and model outputs to ensure sensitive information is never exposed to end users. This reduces privacy risks and helps maintain regulatory compliance (e.g., GDPR, financial data rules).

Secure Storage of Document Embeddings

Embeddings may inadvertently encode sensitive data. They must be stored only in **encrypted vector databases**, with separate encryption keys and strict authorization policies. This prevents unintended disclosure of knowledge encoded in vectors and ensures safe handling of confidential documents.

Role-Based Access Control (RBAC)

RBAC ensures only authorized users or services can ingest data, access vector databases, modify retriever settings, or view logs. Administrators, developers, and support teams should have clearly separated privileges to reduce the risk of misuse and enforce governance policies effectively.

Audit Logging

All ingestion events, retrieval operations, LLM calls, and user interactions should be logged with timestamps and metadata. Logs enable detailed traceability, incident investigations, and compliance reporting. They also help detect anomalies such as unusual query patterns or unauthorized access attempts.

Isolation of Confidential Documents

Highly sensitive documents should be isolated using **separate indexes**, **private vector stores**, or **namespace-level segmentation**. Retrieval filters must ensure that private or regulated data is never returned for general queries. This is critical for domains like banking, insurance, and healthcare.

Enterprise Identity Integration (OAuth/OIDC)

Integrating the system with enterprise authentication providers (OAuth, OIDC, Azure AD, etc.) ensures secure login, centralized identity management, MFA support, and compliance with corporate IT standards. This also enables seamless user provisioning, role assignment, and access revocation.

---

7. Scaling a RAG System (Expanded Multi-Page Section)

Scaling a RAG system is critical when moving from prototype workloads to enterprise-grade, production-level environments. As document volume, query traffic, and latency requirements increase, each part of the RAG pipeline must be optimized and scaled horizontally or vertically. This section explains the major strategies in detail.

7.1 Why Scaling Matters

As your RAG system grows:

- The number of indexed documents increases
- Embedding storage size grows
- Retrieval operations become heavier
- Query rates increase
- LLM latency becomes a bottleneck
- Multi-tenant workloads demand strict SLAs

Scaling ensures:

- Low latency even under heavy load
- High availability (99.9% or 99.99%)
- Cost-efficient resource usage
- Fault tolerance
- Real-time query responses

7.2 Horizontal Scaling of Retrievers

Horizontally scaling retrievers means running multiple retrieval engines in parallel so they can serve many users simultaneously.

How Horizontal Scaling Works

- Deploy multiple retriever instances in containers or VMs
- Use a load balancer to distribute incoming query loads
- Each instance independently queries the vector DB
- Stateless retrieval ensures easy scaling

Why Retrievers Must Be Stateless

To scale effectively, each retriever:

- Should not store conversation history locally
- Should rely on external memory stores (Redis, DynamoDB, MongoDB, PostgreSQL, etc.)

Stateless retrievers = infinite scale.

Deployment Platforms for Scaling

- Kubernetes (HorizontalPodAutoscaler)
- AWS ECS / EKS
- Azure Kubernetes Service
- Google Cloud Run (autoscaling)
- Serverless runtimes (limited but possible)

When to Scale Horizontally

- QPS (queries per second) > 30
- Latency spikes > 300 ms
- When embedding dimension is large (1024–4096)
- When vector DB is remote and network latency matters

7.3 Using Distributed Vector Databases (Milvus, Pinecone, Weaviate)

For systems with millions or billions of vectors, single-node solutions like FAISS or Chroma become insufficient. Distributed vector databases solve this by spreading data across multiple nodes.

Why Use a Distributed Vector Database?

- Handles massive document libraries
- Supports sharding & replication
- Provides high availability
- Offers built-in indexing and search acceleration
- Designed for low-latency retrieval under heavy load

7.3.1 Milvus / Zilliz

Milvus is an open-source, cloud-native vector database optimized for large-scale embeddings.

Features:

- Sharding across compute nodes

- Automatic load balancing
- Multi-replica support
- Supports billion-scale vector storage
- GPU acceleration optional

Best Use Cases:

- Enterprise document stores
- Multimodal embedding search
- Legal/financial institutions with large document bases

7.3.2 Pinecone

Pinecone is a fully managed, distributed vector DB.

Features:

- No infrastructure management
- Dynamic collection scaling
- Pod-based scaling model
- Metadata filtering + hybrid search
- Extremely low latency

Best Use Cases:

- RAG systems with real-time query load
- Startups/enterprises that don't want ops overhead

7.3.3 Weaviate Cloud

Features:

- Distributed architecture
- HNSW-based indices
- Strong metadata filtering
- Multi-tenant environments

7.4 Sharding Strategies (Partitioning the Vector Index)

Sharding divides your vector embeddings across multiple nodes to improve performance.

Types of Sharding:

1. Hash-Based Sharding

- Based on document ID hash
- Ensures uniform distribution
- Good for generalized documents

2. Semantic Sharding (Recommended for RAG)

- Group related domains together
- Example: In banking RAG
    - Loans shard
    - Savings shard
    - Regulations shard
- Improves retrieval accuracy
- Reduces search space

3. Time-Based Sharding

Useful when:

- Documents are appended chronologically (logs, statements)

Sharding Benefits

- Reduces search time
- Improves parallelism
- Allows scaling per topic/domain

7.5 Cache-Based Retrieval Acceleration

Caching dramatically speeds up RAG performance by avoiding repeated vector searches for common queries.

Types of caching:

7.5.1 Query Cache

- Stores frequent queries → retrieved chunks
- Supported by Redis / Memcached
- Ideal for high-traffic applications

Example:

- "What is the interest rate for home loans?" → Cached
- Next user gets instant results

7.5.2 Embedding Cache

- Embeddings for identical or similar queries cached
- Saves LLM embedding generation cost
- Use vector similarity to identify near-duplicates

7.5.3 Chunk Cache

- Cache post-retrieval fused context
- Useful in multi-turn chat flows

7.5.4 LLM Response Cache

- Store final LLM outputs for repetitive questions
- Great for knowledge bases
- Beware of context-dependent questions

7.5.5 Hardware Caching

- GPU memory caching for repeated ANN (Approximate Nearest Neighbor) searches
- NVMe caching for high-read workloads

7.6 Index Optimization

Index optimization ensures your vector search is fast, accurate, and cost-efficient.

7.6.1 Choosing the Right Index Type

Common ANN index types:

- **HNSW** (Hierarchical Navigable Small World)
    - Best accuracy + speed combination
- **IVF** (Inverted File Index)
    - Good for large datasets
    - Requires training
- **PQ / OPQ** (Product Quantization)
    - Reduces memory usage
    - Small accuracy tradeoff

7.6.2 Rebuilding the Index

Indexes degrade over time as new documents are added.

Best practice:

- Rebuild index during maintenance windows
- Or use vector DBs that auto-optimize

7.6.3 Pruning Stale Embeddings

Remove:

- Outdated documents
- Duplicate pages
- Redundant chunks
- Automatically generated noise

Pruning reduces:

- Search latency
- Memory usage
- Storage costs

7.6.4 Metadata-Based Filters

Always use metadata to reduce search space:

Filters examples:

- document_type
- department
- page_range
- category
- version

Example:

filter: { “category”: “KYC Policy” }

This gives a **10× faster search** because it avoids scanning unrelated embeddings.

7.6.5 Hybrid Search Optimization

Hybrid = vector search + BM25 keyword search

Benefits:

- Improves accuracy
- Helps with short queries
- Helps with acronyms / technical jargon

7.7 Scaling the LLM Layer

Scaling the LLM handling layer is equally important.

Strategies:

- Use streaming LLM responses
- Use model multiplexing (switch to smaller models for simple queries)
- Distribute load across multiple inference servers
- Cache final responses
- Fine-tune model for domain to reduce prompt size

---

8. Testing a RAG System

1. LangSmith (Testing + Tracing)

LangSmith provides datasets, evaluators, and detailed traces for RAG applications. It lets you test retrieval quality, answer relevance, and accuracy using structured evaluation workflows. It also records step-by-step traces of RAG runs for debugging.

2. Langfuse + Ragas (Observability + Scoring)

Langfuse adds tracing, span-level metrics, and dashboards, while Ragas provides reference-free RAG metrics like *faithfulness*, *answer relevancy*, and *context precision*. You can evaluate real production conversations without needing ground-truth labels.

3. Arize Phoenix (RAG Evaluations)

Phoenix supports retrieval metrics like *NDCG\@k, Precision\@k, Hit Rate*, and **answer correctness/hallucination evaluators*. It integrates cleanly with LlamaIndex pipelines for RAG debugging and quality monitoring.

4. TruLens (Feedback + RAG Triad)

TruLens evaluates RAG using the RAG Triad:

- *Context Relevance* (retrieval)
- *Groundedness* (faithfulness)
- *Answer Relevance* (generation)
  It uses "feedback functions" and OpenTelemetry-compatible tracing for comparing versions and diagnosing failure points.

5. Promptfoo (Testing + Red Teaming)

Promptfoo enables declarative test suites for RAG (factuality, context adherence, relevance) and supports security testing such as prompt injection and context injection. It is ideal for CI/CD automation.

6. Helicone (Cost Tracking + Monitoring)

Helicone acts as a proxy to log token usage, cost, latency, and lets you segment costs by *user*, *feature*, or *session*. It supports optimization via caching and model routing.

7. Weights & Biases (Experiment Tracking)

W\&B Weave automatically logs LLM calls, token cost, latency, and errors, helping compare prompt/model variations during RAG development. It provides dashboards for performance vs cost.

---

9. Prompt Engineering for RAG Development (Amazon Q + Others)

Prompt engineering plays a key role when using AI tools such as Amazon Q Developer, Azure AI Studio, or OpenAI assistants to auto-generate code, backend workflows, frontend UI components, deployment files, and security configurations. The prompts below illustrate how to direct AI to create a production-grade RAG system with clear instructions about the language, frameworks, libraries, folder structure, and security practices.

9.1 Prompt for Building the Entire RAG Application (Fully Detailed)

Use this prompt with Amazon Q, Azure AI, or any GenAI coding assistant:

Prompt:
Build a complete RAG-based application with the following requirements:

1. Document Ingestion Pipeline

- Input: PDF files
- Steps: extract → chunk → embed → store in vector DB
- Use <embedding_model> for both indexing and queries
- Include code for text extraction, cleaning, and chunking

2. Backend Language & Framework

- Backend must be implemented in: **<language> (e.g., Java, Python, Node.js)**
- If Java: use **Spring Boot + LangChain4j + Pinecone/Weaviate/Milvus client**
- If Python: use **FastAPI + LangChain + Chroma/Pinecone/Milvus**
- Provide REST endpoints:
    - POST /ingest – upload documents
    - POST /chat – RAG query endpoint
    - GET /health
    - GET /stats

3. Vector Database

- Use <vector_db> (e.g., Pinecone, Weaviate, Milvus, Chroma)
- Show code to create index, insert embeddings, query top-k results
- Include metadata fields: filename, section, page, timestamp

4. LLM Integration

- Use <selected_model> (e.g., GPT-4, Claude, Gemini, Azure OpenAI)
- Show system prompt, user prompt, RAG augmentation format
- Include streaming response support if available

5. Frontend UI

- Build the UI using **Angular or React**
- Include:
    - Chat interface
    - Conversation history
    - Loading indicators
    - Citation display of retrieved documents
    - Error handling (timeouts, network errors)

6. Security Requirements

- Implement **RBAC** using JWT or OAuth2
- Roles: USER, ADMIN
- Admin can ingest documents; User can only query
- Add input validation, rate limiting, and CORS configuration
- Use HTTPS-only configuration and secure cookies (if applicable)

7. Folder Structure & Project Setup

- Generate a complete folder layout for backend + frontend + deployment
- Include README with setup instructions

8. Docker Support & Deployment

- Create Dockerfile for backend and frontend
- Create docker-compose.yml that includes:
    - Vector DB service
    - Backend service
    - Frontend service
- Configure environment variables for keys (LLM keys, DB keys)
- Add CI/CD notes for GitHub Actions or AWS CodePipeline

Provide complete code, explanations, comments, and best practices.

9.2 Prompt for Creating a Vector Database Index

Prompt:
Generate backend code in **<language>** that creates a vector index in **<vector_db>** using the embedding model **<embedding_model>**.

Requirements:

- Initialize vector DB client
- Create index with desired dimensions and metadata schema
- Implement /ingest endpoint:
    - Accept PDF file
    - Extract → chunk → embed using <embedding_model>
    - Upsert vectors into the index
- Return ingestion status and document metadata

Include environment variable usage, error handling, and logging.

9.3 Prompt for Adding Chat History

Prompt:
Modify the RAG chat endpoint to include **conversation memory**.

Requirements:

- Maintain last 5 user messages + last 5 assistant messages
- Add memory to the RAG prompt before calling the LLM
- Use Redis, database storage, or in-memory session store
- Provide updated endpoints:
    - /chat → accepts conversationId
    - Auto-load history and attach to the prompt
- Ensure history does NOT leak sensitive info (apply PII protection)

Update both backend logic and frontend UI to render the chat timeline.

9.4 Prompt for Deployment Scripts

Prompt:
Generate deployment scripts for the entire RAG application.

1. Dockerfile for Backend

- Install <language> runtime
- Install needed libraries (LangChain, LangChain4j, Spring Boot dependencies, etc.)
- Expose default port
- Use multi-stage build for optimization

2. Dockerfile for Frontend (Angular/React)

- Install Node
- Build static assets
- Serve via NGINX
- Add CSP headers

3. docker-compose.yml

- Service: vector DB (Pinecone/Weaviate/Milvus/Chroma)
- Service: backend
- Service: frontend
- Service: Redis (optional for memory)
- Add volume mounts
- Add environment variables
- Add network configuration

4. Security & Environment Management

- All secrets must stay in .env
- Show example .env file with placeholders
- Enforce RBAC roles in the backend
- Configure HTTPS (reverse proxy or AWS ALB)

5. CI/CD Instructions

- Generate GitHub Actions YAML to build & push Docker images
- Add linting, tests, and RAG eval test stage
- Add deployment step to AWS ECS/EKS or Azure App Service

Provide all configs with explanations.

9.5 Optional Prompt: Add Security, RBAC, & Governance Requirements

Prompt:
Enhance the backend with strong security, privacy, and governance:

- Implement **JWT authentication** with roles: ADMIN, USER
- Only ADMIN can ingest documents; USER can only chat
- Add PII masking before indexing and before response output
- Encrypt embedding storage if vector DB supports it
- Log all ingestion & retrieval events
- Add rate limiting
- Add input validation for all endpoints
- Add CORS rules for the frontend domain only
- Comply with OAuth/OIDC integration using <provider> (Azure AD, Okta, AWS Cognito)

Generate updated code showing controller, security config, filters, and middleware.

9.6 Optional Prompt: UI Security Requirements (Angular or React)

Prompt:
Update the UI with security features:

- Login page integrated with backend JWT/OIDC
- Store access tokens securely (httpOnly cookie or memory storage)
- Restrict Admin Panel (document ingestion page)
- Show RBAC-based menu items
- Add CSRF protection
- Add sanitization for user input
- Add UI for viewing citations & source documents

---

10. Banking Domain Example Use Case

Objective

To build a domain-specialized RAG system that supports banking operations by delivering accurate, policy-grounded answers to employees and customers. The objective is to improve

compliance, reduce manual workload, and ensure consistency across branch operations by grounding all responses in official banking documents.

Business Context

Banks manage large volumes of regulatory documents, loan policies, KYC rules, and interest-rate sheets that frequently change. Frontline staff often rely on outdated knowledge or manual searches, creating compliance risks and operational delays. A RAG-based solution helps unify access to critical banking information, enabling faster decision-making, accurate customer service, and reduced dependency on physical manuals or senior staff verification.

Use Case Coverage

- **Loan Policy Manual RAG:** Retrieve eligibility rules, documentation requirements, limits, and conditions directly from loan manuals.
- **Interest Rate Queries:** Provide updated and segmented rate information based on product, customer type, or tenure.
- **KYC Guidelines:** Offer precise compliance instructions for various customer categories (resident, NRI, corporate, senior citizen).
- **Document Linking With Metadata:** Enable query-specific retrieval with section-level references, timestamps, and document categories for auditability.
- **Customer Attention & Verification RAG:** Assist in verifying requirements, guiding customers through banking processes, and clarifying eligibility conditions.

Deliverables

- A fully indexed vector database of banking manuals, KYC rules, and product documents.
- A retrieval-augmented chatbot/interface for staff or customers.
- Metadata-enhanced retrieval that cites source documents for compliance.
- Automated workflows supporting loan processing, KYC verification, and customer guidance.
- Audit-ready responses grounded in approved policies to minimize compliance risk.

11. Troubleshooting Guide

Mismatched Embeddings

Using different embedding models for ingestion and querying results in inaccurate similarity scores and irrelevant retrievals. Always use the same embedding model and tokenizer. Re-embed and re-index documents after upgrading or changing the embedding model.

Prompt to Fix (Give to AI Assistant):

"Analyze my RAG pipeline and confirm whether the embedding model used in ingestion and querying is identical. If not, generate code to re-embed documents and rebuild the vector index using <embedding_model>."

Retrieval Failing Due to Poor Chunking

Chunk sizes that are too small lose context, while oversized chunks dilute meaning. This causes low retrieval quality and incomplete answers. Ideal chunk sizes range between 300–1200 tokens with appropriate overlap.

Prompt to Fix:

"Review my chunking settings and propose an optimized chunk size and overlap for my dataset. Then regenerate the document-splitting code using <chunking_strategy> to improve retrieval accuracy."

LLM Hallucinating

Hallucinations occur when retrieved context is irrelevant or insufficient. Improve chunking, enhance metadata filters, tighten your system prompt, and ensure retrieved sources are passed clearly into the prompt.

Prompt to Fix:

"Inspect my RAG prompt template and retrieval function. Update them to reduce hallucination by adding grounding instructions, improving context injection, and enforcing citation of retrieved chunks."

Memory Overflow

Storing too much conversation history or overly large retrieved text overwhelms the LLM's context window. Implement sliding-window memory or summarization of older messages.

Prompt to Fix:

“Analyze my chat history implementation and modify it to use a sliding-window memory that only keeps the last N interactions. Add optional summarization for older messages.”

Index Corruption

Index corruption can occur due to abrupt shutdowns, schema changes, or faulty updates. Maintain backups, snapshots, and consider rebuilding the index if corruption is detected.

Prompt to Fix:

“Check my vector index for schema mismatches or corrupt entries. If issues are found, generate instructions and code to rebuild the index from the original documents.”

Slow Retrieval

Large indexes, unoptimized ANN search parameters, or missing metadata filters slow retrieval. Enable ANN indexing (HNSW, IVF), add metadata filtering, increase vector DB resources, and consider caching frequent queries.

Prompt to Fix:

“Optimize my vector database retrieval by tuning ANN parameters, enabling metadata filters, and adding caching for repeated queries. Generate the updated vector DB configuration steps.”

BONUS: A Single Master Prompt for the Entire Troubleshooting Section

If you want your AI assistant to automatically detect and fix **any** of the above issues:

Master Prompt:
“I am building a RAG application. Analyze my codebase and identify issues related to embeddings, chunking, retrieval quality, hallucinations, memory limits, vector index structure, and retrieval latency. For each problem found, generate corrected code, updated configurations, and best-practice recommendations.”

12. Final Recommendations

Choosing the right design patterns and operational practices is essential for building a reliable, scalable, and high-quality RAG system. The following recommendations serve as best practices for maintaining consistency, improving retrieval accuracy, and ensuring long-term stability.

Choose Embeddings Wisely

Select an embedding model that aligns with the domain, content type, and performance requirements. Consistency is critical—use the same embedding model for both document ingestion and query embedding to avoid retrieval mismatches.

Use Metadata-Based Filtering

Enhance precision by leveraging metadata such as section names, categories, timestamps, or document types. Metadata filters help narrow down retrieval results and reduce noise, especially in large document collections.

Keep Chunk Sizes Consistent

Maintain consistent chunk sizes throughout ingestion to ensure predictable retrieval behavior. Balanced chunk sizes (e.g., 300–1200 tokens) provide enough context without overwhelming the model or causing fragmented answers.

Use Hybrid Search for Better Recall

Combine semantic search with keyword or sparse search techniques to improve retrieval coverage. Hybrid strategies ensure that both meaning-based and keyword-specific queries return relevant results.

Add Memory If Building a Chatbot

For conversational applications, implement a short-term memory system to preserve context across turns. Use sliding windows or summarized memory to avoid exceeding token limits while maintaining conversation coherence.

Log Every Stage

Enable detailed logging for ingestion, embedding, retrieval, and LLM inference stages. Logs support debugging, audit requirements, performance analysis, and monitoring of unusual behavior in production systems.

Version Your Indices

Maintain versioned vector indexes to ensure rollbacks, reproducibility, and safe experimentation. Index versioning allows you to test new embedding models or chunking strategies without affecting live systems.